Lab 22 – Advanced Ethical Considerations: Addressing Complex Scenarios and Limitations

Task 1 – Bias Detection in AI Suggestions (Alternative Example)

Task: Ask AI to generate a resume shortlisting algorithm multiple times. Change prompt wording (e.g., “select best candidate” vs “ensure diversity”). Compare outputs and logic.

Discussion Points:

- Does the AI favor certain qualifications, backgrounds, or experiences?
- How could biased shortlisting algorithms affect equal employment opportunities?

Prompt

> "Write a Python script that simulates a resume shortlisting algorithm. First, create a function that optimizes for the 'best candidate' based purely on strict numerical metrics (like GPA and years of experience). Then, write a second function designed to 'ensure diversity' by ensuring equal representation from different backgrounds. Print the selected candidates from a mock dataset to compare the logic."

Code

```python
import pandas as pd

# Mock Candidate Data
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
    'GPA': [3.9, 3.8, 3.2, 3.5, 3.1],
    'Experience_Yrs': [5, 4, 2, 3, 1],
    'Background': ['Tech Hub', 'Tech Hub', 'Underrepresented', 'Tech Hub', 'Underrepresented']
}
df = pd.DataFrame(data)

def shortlist_best(df, top_n=2):
    """AI's typical interpretation of 'Best': strict descending sort on metrics."""
    return df.sort_values(by=['GPA', 'Experience_Yrs'], ascending=[False, False]).head(top_n)

def shortlist_diverse(df, top_n=2):
    """Adjusted logic to 'Ensure Diversity' across backgrounds."""
    # Select top candidate from each unique background group
    diverse_picks = df.groupby('Background').apply(
        lambda x: x.sort_values(by=['GPA'], ascending=False).head(1)
    ).reset_index(drop=True)
    return diverse_picks.head(top_n)

print("--- 'Best Candidate' Shortlist ---")
print(shortlist_best(df).to_string(index=False))

print("\n--- 'Ensure Diversity' Shortlist ---")
print(shortlist_diverse(df).to_string(index=False))
```

Output

```
PS C:\Users\nikhi\Projects\AI Assistant Coding> python .\Assignment-22.py
--- 'Best Candidate' Shortlist ---
 Name  GPA  Experience_Yrs Background
Alice  3.9               5   Tech Hub
  Bob  3.8               4   Tech Hub

--- 'Ensure Diversity' Shortlist ---
   Name  GPA  Experience_Yrs
  Alice  3.9               5
Charlie  3.2               2
```

Explanation

When asked for the "best" candidate, the AI defaults to strict numerical sorting, which often favors individuals with privileged access to resources, inadvertently filtering out minority groups. Tweaking the logic to prioritize diversity fundamentally changes the output,

highlighting how hidden algorithmic biases can severely impact equal employment opportunities if left unchecked.

Task 2 – Privacy and Sensitive Data Handling (Alternative Example)

Task: Ask AI to write code for processing student academic records (roll number, marks, email).

Discussion Points:

- How should student identity data be masked or encrypted?
- What privacy laws (e.g., FERPA, GDPR-like principles) are relevant?
- What ethical responsibility does the programmer have when using AI-generated code?

Prompt

> "Generate a Python script to process student academic records containing roll numbers, marks, and emails. Implement data masking to hide the sensitive parts of the emails and roll numbers to comply with privacy principles like FERPA/GDPR. Display the original vs. masked data."

Code

```python
import re

students = [
    {"roll": "23CSBTB01", "name": "John Doe", "marks": 88, "email": "johndoe@university.edu"},
    {"roll": "23CSBTB02", "name": "Jane Smith", "marks": 92, "email": "janesmith@university.edu"}
]

def mask_sensitive_data(record):
    """Masks Roll Number and Email for privacy compliance."""
    masked_record = record.copy()

    # Mask Roll Number: Keep only last 2 digits
    original_roll = record['roll']
    masked_record['roll'] = "********" + original_roll[-2:]

    # Mask Email: Keep first letter, mask middle, keep domain
    email = record['email']
    user, domain = email.split('@')
    masked_record['email'] = f"{user[0]}***@{domain}"

    # Remove name entirely for anonymity in reporting
    del masked_record['name']

    return masked_record

print("--- Original Data (CONFIDENTIAL) ---")
print(students[0])

print("\n--- Masked Data (FERPA/GDPR Compliant for Logging) ---")
masked_students = [mask_sensitive_data(s) for s in students]
print(masked_students[0])
```

Output

```
PS C:\Users\nikhi\Projects\AI Assistant Coding> python .\Assignment-22.py
--- Original Data (CONFIDENTIAL) ---
{'roll': '23CSBTB01', 'name': 'John Doe', 'marks': 88, 'email': 'johndoe@university.edu'}

--- Masked Data (FERPA/GDPR Compliant for Logging) ---
{'roll': '********01', 'marks': 88, 'email': 'j***@university.edu'}
PS C:\Users\nikhi\Projects\AI Assistant Coding>
```

Explanation

When processing sensitive student data, programmers have an ethical and legal duty (under laws like FERPA or GDPR) to protect user identity. This script demonstrates the programmer's responsibility to proactively apply data masking techniques to AI-generated pipelines, ensuring identifiable information is never exposed in analytics or system logs.

Task 3 – AI-Generated Security Risks (Alternative Example)

Task: Use AI to generate a payment processing script for an online shopping system.

Identify risks such as: Plain-text card data storage, Missing input validation.

Propose ethical coding standards to prevent financial fraud and misuse.

Prompt

> "Write a Python snippet for a vulnerable payment processing script that stores card data in plain text and lacks input validation. Then, provide a refactored secure version that uses a mock payment tokenization system and validates the input, discussing the ethical coding standards required to prevent fraud."

Code

```python
# --- VULNERABLE AI-GENERATED CODE ---
class VulnerablePaymentProcessor:
    def process_payment(self, card_number, amount):
        # RISK: No input validation. Plain-text storage.
        with open("insecure_logs.txt", "a") as f:
            f.write(f"Charged {card_number} amount ${amount}\n")
        print(f"[Vulnerable] Charged ${amount} to {card_number}")

# --- SECURE REFACTORED CODE ---
class SecurePaymentProcessor:
    def process_payment(self, payment_token, amount):
        # ETHICAL STANDARD: Input validation
        if amount <= 0:
            raise ValueError("Invalid payment amount.")
        if not payment_token.startswith("tok_"):
            raise ValueError("Invalid payment token. Raw card numbers are forbidden.")

        # ETHICAL STANDARD: Never log sensitive data; rely on secure tokens (PCI-DSS compliance)
        print(f"[Secure] Transaction of ${amount} processed using secure token: {payment_token[:8]}***")

# Testing the implementations
insecure = VulnerablePaymentProcessor()
insecure.process_payment("1234-5678-9012-3456", 150.00)

secure = SecurePaymentProcessor()
try:
    # Simulating tokenized payment (raw cards never hit our server)
    secure.process_payment("tok_abc123xyz987", 150.00)
except ValueError as e:
    print(f"Error: {e}")
```

Output

```
PS C:\Users\nikhi\Projects\AI Assistant Coding> python .\Assignment-22.py
[Vulnerable] Charged $150.0 to 1234-5678-9012-3456
[Secure] Transaction of $150.0 processed using secure token: tok_abc1***
```

Explanation

AI frequently generates naive code that blindly stores credit card numbers in plain text, violating PCI-DSS regulations. By identifying these flaws and refactoring the script to strictly use payment tokens and validate inputs, we uphold the ethical coding standards necessary to prevent severe financial fraud and protect consumer trust.

Task 4 – Accountability in Human–AI Collaboration (Alternative Example)

Scenario: AI generates code for an autonomous traffic signal control system.

Discussion Questions:

- If a software error causes accidents, who is accountable?
- Should responsibility lie with AI tool developers, Software engineers, or Government authorities deploying the system?
- Students submit a short reflection on accountability.

Prompt

> "Provide a conceptual Python script for an autonomous traffic signal control system. Include a printed reflection text that answers the question: If a software error in this AI-generated code causes a traffic accident, who is ultimately accountable (AI developers, software engineers, or deploying authorities)?"

Code

```python
def autonomous_traffic_control(sensor_data):
    """ (parameter) sensor_data: Any control logic."""
    if sensor_data['cars_waiting_north'] > 10:
        return "GREEN_NORTH_SOUTH"
    return "RED_ALL"

print("--- System Status ---")
print(f"Traffic Lights set to: {autonomous_traffic_control({'cars_waiting_north': 15})}")

print("\n--- Reflection on Accountability ---")
reflection = """
If a software error in an AI-generated traffic system causes an accident, the accountability
cannot be placed on the AI tool or its developers, as AI is merely an assistive utility.
The primary responsibility lies with the SOFTWARE ENGINEERS who integrated the code without
sufficient testing and fail-safes. Furthermore, the GOVERNMENT AUTHORITIES deploying the
system share the liability, as they are responsible for certifying that safety-critical
public infrastructure undergoes rigorous human oversight, simulation, and quality assurance
before going live.
"""
print(reflection.strip())
```

Output

```
PS C:\Users\nikhi\Projects\AI Assistant Coding> python .\Assignment-22.py
--- System Status ---
Traffic Lights set to: GREEN_NORTH_SOUTH

--- Reflection on Accountability ---
If a software error in an AI-generated traffic system causes an accident, the accountability
cannot be placed on the AI tool or its developers, as AI is merely an assistive utility.
The primary responsibility lies with the SOFTWARE ENGINEERS who integrated the code without
sufficient testing and fail-safes. Furthermore, the GOVERNMENT AUTHORITIES deploying the
system share the liability, as they are responsible for certifying that safety-critical
public infrastructure undergoes rigorous human oversight, simulation, and quality assurance
before going live.
```

Explanation

In life-critical systems like traffic control, ultimate accountability cannot be offloaded to an AI assistant. The human software engineers and the deploying authorities bear the legal and ethical responsibility to rigorously test, validate, and monitor the AI-generated logic before putting public lives at risk.

Task 5 – Limitations of AI in Critical Systems (Alternative Example)

Scenario: Ask AI to generate code for an aircraft maintenance fault detection system.

Discussion Points:

- Can AI-generated code be trusted without expert validation?
- What human oversight is essential in aviation software?
- How should safety-critical code be reviewed and certified?

Prompt

> "Generate a conceptual Python script for an aircraft maintenance fault detection system. The script should explicitly require a manual override or verification step. Print a warning log discussing why AI-generated code cannot be trusted blindly and what human oversight/certification is required in aviation software."

Code

```python
class AircraftFaultDetector:
    def __init(self):
        self.ai_confidence = 0.95

    def analyze_engine_sensors(self, sensor_readings):
        print("Analyzing engine telemetry using AI models...")
        # Simulated AI logic
        if sensor_readings.get("vibration_level") > 0.8:
            print("[AI ALERT] High vibration detected. Potential turbine fault.")
            self.require_human_validation()

    def require_human_validation(self):
        print("\n" + "="*50)
        print("CRITICAL LIMITATION WARNING: HUMAN OVERRIDE REQUIRED")
        print("="*50)
        print("AI-generated diagnostics CANNOT be trusted blindly in aviation.")
        print("Essential Human Oversight:")
        print("1. A certified aviation mechanic MUST manually inspect the turbine.")
        print("2. Safety-critical software must be reviewed and certified under DO-178C standards.")
        print("3. AI is a support tool; the final airworthiness decision is strictly human.")
        print("="*50)

# Simulate a diagnostic run
detector = AircraftFaultDetector()
detector.analyze_engine_sensors({"vibration_level": 0.85})
```

Output

```
PS C:\Users\nikhi\Projects\AI Assistant Coding> python .\Assignment-22.py
Analyzing engine telemetry using AI models...
[AI ALERT] High vibration detected. Potential turbine fault.

==================================================
CRITICAL LIMITATION WARNING: HUMAN OVERRIDE REQUIRED
==================================================
AI-generated diagnostics CANNOT be trusted blindly in aviation.
Essential Human Oversight:
1. A certified aviation mechanic MUST manually inspect the turbine.
2. Safety-critical software must be reviewed and certified under DO-178C standards.
3. AI is a support tool; the final airworthiness decision is strictly human.
==================================================
```

Explanation

Aviation software operates under zero-margin-for-error conditions, making raw AI-generated code or diagnostics far too risky to deploy autonomously. Because AI lacks real-world contextual understanding, safety-critical code must undergo stringent manual review, strict certification processes (like DO-178C), and expert human validation to ensure absolute passenger safety.

Task 6 – Ethical Use of AI-Generated Code (Alternative Example)

Scenario: Students analyze cases where AI-generated code is used in academic assignments or startups.

Discussion Points:

- Risks of submitting AI code without understanding it
- Copyright and license violations
- Best practices: Proper citation, Manual verification, Ethical reuse of AI outputs

Prompt

> "Write a Python script that acts as an 'Ethical Code Linter'. It checks a mock source code string for an AI citation comment. If missing, it prints a warning discussing the risks of plagiarism, copyright violations, and the best practices for ethical reuse of AI outputs in academia and startups."

Code

```python
def ethical_code_linter(source_code):
    print("Running Ethical AI Linter...")

    # Check for proper attribution
    if "Generated by AI" in source_code or "Citation:" in source_code:
        print("[PASS] Proper AI citation found. Proceed with manual verification.")
    else:
        print("\n[FAIL] Missing AI Citation! Review Ethical Guidelines Below:")
        print("-" * 55)
        print("1. Risks of Blind Submission: Submitting code you don't understand ")
        print("   leads to academic misconduct, failing grades, and introduces ")
        print("   unmaintainable technical debt in startups.")
        print("2. Copyright Violations: AI models are trained on public repos.")
        print("   Unmodified AI code may infringe on existing open-source licenses.")
        print("3. Best Practices: Always provide PROPER CITATION (e.g., 'Adapted ")
        print("   from ChatGPT'), perform MANUAL VERIFICATION to ensure correctness,")
        print("   and use AI for logic assistance rather than wholesale copying.")
        print("-" * 55)

# Mock code submitted by a student
mock_student_submission = """
def quick_sort(arr):
    if len(arr) <= 1: return arr
    pivot = arr[len(arr) // 2]
    # No citation provided...
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
"""

ethical_code_linter(mock_student_submission)
```

Output

```
PS C:\Users\nikhi\Projects\AI Assistant Coding> python .\Assignment-22.py
Running Ethical AI Linter...

[FAIL] Missing AI Citation! Review Ethical Guidelines Below:
-----------------------------------------------------
1. Risks of Blind Submission: Submitting code you don't understand
   leads to academic misconduct, failing grades, and introduces
   unmaintainable technical debt in startups.
2. Copyright Violations: AI models are trained on public repos.
   Unmodified AI code may infringe on existing open-source licenses.
3. Best Practices: Always provide PROPER CITATION (e.g., 'Adapted
   from ChatGPT'), perform MANUAL VERIFICATION to ensure correctness,
   and use AI for logic assistance rather than wholesale copying.
-----------------------------------------------------
```

Explanation

Blindly copying AI code for academic assignments or commercial products can lead to severe consequences like plagiarism, copyright infringement, and introducing unknown vulnerabilities. Ethical reuse requires programmers to thoroughly understand the generated logic, manually verify its safety, and provide proper attribution to maintain academic and professional integrity.